

Documentation

Birthday Cake Candles

Algorithm Analysis & Complexity Comparison

First Algorithm — Brute Force

Pseudocode

```
Algorithm BirthdayCakeCandles (Candles[n], n){

    Counter ← 0                →(1)
    Max ← -1                    →(1)

    For i ← 0 to n-1 do        →(n+1)
        If Max < Candles[i] Then →(n)
            Max ← Candles[i]    →(n)

    For i ← 0 to n-1 do        →(n+1)
        If Max == Candles[i] Then →(n)
            Counter ← Counter + 1 →(n)

    Return Counter            →(1)
}
```

Calculation of $T(n)$

$$T(n) = 1 + 1 + (n+1) + n + n + (n+1) + n + n + 1$$

$$\text{Constants: } 1+1+1+1+1 = 5$$

$$n \text{ terms: } n+n+n+n+n+n = 6n$$

$T(n)$	$6n + 5$
--------	----------

Time Complexity	$O(n)$ — Linear (growth rate: highest power of n is 1)
-----------------	--

Second Algorithm — Decrease and Conquer (Recursive)

Pseudocode

```

ALGORITHM BirthdayCakeCandles(candles, n, index, maxH, count){

    If index == n Then                                →(1)
        Return count                                  →(1)

    If candles[index] > maxH Then                      →(1)
        maxH ← candles[index]                        →(1)
        count ← 1                                     →(1)

    Else If candles[index] == maxH Then                →(1)
        count ← count + 1                            →(1)

    Return BirthdayCakeCandles(candles, n, index+1, maxH, count) →T(n-1)
}

```

Recurrence & Substitution

Step 1: $T(n) = T(n-1) + c$

Step 2: $T(n) = T(n-2) + 2c = T(n-3) + 3c = T(n-k) + kc$

$k = n-1 \rightarrow T(n) = T(1) + (n-1)c = c \cdot n$

Step 3: Final Result

Time Complexity **$O(n)$** **Third Algorithm — Divide and Conquer (Recursive)****Pseudocode**

```

Algorithm BirthdayCakeCandles(candles, low, high){

    If low == high Then                                →(1)
        maxH ← candles[low]                          →(1)
        Return 1                                     →(1)

    mid ← (low + high) / 2                            →(1)

    left_count ← BirthdayCakeCandles(candles, low, mid, left_max) →T(n/2)
    right_count ← BirthdayCakeCandles(candles, mid+1, high, right_max) →T(n/2)

    If left_max > right_max Then                      →(1)
        maxH ← left_max                              →(1)
        Return left_count                            →(1)

    Else If right_max > left_max Then                →(1)

```

```

    maxH ← right_max
    Return right_count                                → (1)

Else                                                → (1)
    maxH ← left_max                                → (1)
    Return left_count + right_count                → (1)
}

```

Recurrence & Master Theorem

Step 1: $T(n) = 2T(n/2) + c$

Step 2: $T(n) = aT(n/b) + f(n) \rightarrow a=2, b=2, f(n)=c$

Step 3: $n^{\log_b a} = n^{\log_2 2} = n^1$ vs $f(n) = c = n^0$

$n^1 > n^0 \rightarrow$ Case 1 of Master Theorem

Time Complexity	$O(n)$
------------------------	--------------------------

Algorithms Comparison

All three algorithms share the same complexity in all 3 cases (Best = Average = Worst = $O(n)$)

Comparison Criteria	Brute Force (Iterative)	Decrease & Conquer (Recursive)	Divide & Conquer (Recursive)
Time Analysis	$6n + 5$	$T(n-1) + c$	$2T(n/2) + c$
Time Complexity	$O(n)$	$O(n)$	$O(n)$
Space Complexity	$O(1)$	$O(n)$	$O(\log n)$
Performance	Fastest	Slowest	Average
Implementation	Loops	Function calls & Base case	Function calls & Base case
Robustness (Stability)	Stable (Safe)	Unstable (High Risk)	Average (Safe for typical n)
Execution Type	Serial	Serial	Parallel
Description	Input independent	Recursion overhead (causes stack overflow)	Recursion Overhead (causes stack overflow; less than Decrease & Conquer)